

Exam: B188633

Date: March 21, 2022

Two-Dimensional Decomposition of Percolate Message Passing Parallel Program

Course: Message Passing Programming

Course Organiser: Dr David Henty

Assessment: 1



THE UNIVERSITY *of* EDINBURGH

Table of Contents

1. Introduction.....	2
2. Proposed Changes and Use of MPI.....	2
2.1 Two-dimensional decomposition.....	2
2.2 Non-blocking communications	2
2.3 New boundary conditions.....	2
2.4 IO.....	3
2.5 Average value.....	3
2.6 Stopping computation	3
2.7 Other possible changes.....	4
3. Program Structure.....	4
3.1 Library for communications.....	4
3.2 Library for core computational routines.....	4
3.3 Other possible changes.....	5
3.4 Concept map.....	5
4. Testing.....	5
4.1 Correctness testing	5
4.2 Performance testing	6
5. Conclusion	8
References.....	8
Appendix A: Correctness Tests.....	9

1. Introduction

In this report, we discuss a plan to parallelise the percolate problem, described in [1], using two-dimensional decomposition. We have a working message-passing parallel solution to the percolate problem implemented in the C programming language, where we use a one-dimensional decomposition. So, we focus on what we need to change to transform the existing parallel code to the final version using two-dimensional decomposition.

2. Proposed Changes and Use of MPI

In this section, we discuss the essential changes needed to transform our existing parallel code using one-dimensional decomposition to the final version using two-dimensional decomposition. Also, we discuss the usage of various routines of the Message Passing Interface (MPI) wherever necessary.

2.1 Two-dimensional decomposition

A simple implementation to decompose the $L \times L$ grid into smaller two-dimensional grids of size $M \times N$ is to divide L by P (i.e., number of processes) for both dimensions. We will check that P divides L exactly. Otherwise, we will return an error message to the user and abort the program.

Our approach to decompose the grid is simple, fractal, easy to implement, and ensures load balance across processes. However, if time allows, we may implement a more complex approach where the user can run the program with any configuration of processes, even with a prime number of processes.

2.2 Non-blocking communications

We will use non-blocking communications for exchanging halos with neighbouring processes (i.e., up, down, left, and right). With this approach, we can overlap communication and computation to make our program more efficient. The cells requiring halos would be computed after the communications complete. We will use a combination of `MPI_Isend()` and `MPI_Recv()` to achieve this instead of the blocking `MPI_Sendrecv()` routine that is used in the current version. `MPI_Waitall()` will be called to ensure that the non-blocking communications with the neighbours complete.

2.3 New boundary conditions

We will implement a new boundary condition, which will be periodic in the horizontal direction. The current version is non-periodic in the horizontal direction. We will use a two-dimensional cartesian virtual topology to compute neighbours and set boundary condition to periodic in the horizontal

direction. We will use **MPI_Cart_create()** to create the two-dimensional topology. We will call **MPI_Cart_shift()** to compute the neighbouring processes. The new boundary condition will optimise the performance of our program as large numbers can travel both left and right periodically in the horizontal dimension. This will slightly change the shape of the clusters in the output image.

2.4 IO

A simple implementation of IO in the two-dimensional version would be to send the whole map array to other processes, then the individual processes can copy back its relevant part into its smaller local map array. With this approach, it will be straightforward to use **MPI_Bcast()** and **MPI_Reduce()** to send and receive the data respectively by the root process.

The current parallel version uses a combination of **MPI_Scatter()** and **MPI_Gather()**. Although this is simple to use in the one-dimensional version, it will be difficult to generalise this approach to the two-dimensional version. This is due to not all the data being contiguous in a two-dimensional grid. So, if we use a vector to send data, will have to offset the starting point at irregular intervals, which makes the logic difficult and cumbersome to implement.

2.5 Average value

We will compute and print the average value of the map array at regular intervals in the iterative loop. A simple approach to implement this would be to place this code above the print statement in the current parallel version of the number of changes at regular intervals. The individual processes can add the values of the cells of local grid (i.e., without the halos). Then, we will call **MPI_Allreduce()** to compute the global sum of the grid. Then, the root process can divide the global sum by the total number of cells (i.e., $L \times L$) to get the average value, and then print the value.

2.6 Stopping computation

The current parallel version runs the core iterative update loop for a fixed number of steps. This is not necessary to do because in many configurations, the algorithm completes before the steps are completed. So, the code is repetitively executing more steps even though there are no further changes. In our final version, at the end of every iteration, we will check whether the number of changes in the iteration is zero; if it is zero, we will break out of the while loop.

For performance testing, we will keep another version of the while loop where there is a fixed number of steps. This is because in a small grid size, we may have to increase the number of steps so that the algorithm runs for a reasonable amount of time to reduce any noise from the system. For a large grid size configuration, we do not need to complete the algorithm as it will unnecessarily burn CPU time.

2.7 Other possible changes

The above subsections discussed the essential changes needed to fulfil the core requirements of our program. However, if time allows, there are additional changes we could implement to make our program further elegant, performant, flexible, and extensible. For instance, we could take the values of P , L , and ρ as runtime arguments as opposed to the compile time arguments where these values are currently defined. Although not mission critical, we may attempt to make the IO more elegant.

3. Program Structure

In this section, we discuss how we will structure the program to make it elegant, readable, maintainable, flexible, and extensible.

3.1 Library for communications

We will abstract all MPI routines away from the main function into a separate communications library. With this approach, we can declutter the main function from the verbose details of the MPI routines (e.g., communicator, tag, datatype). The main function will call routines from the communications library with less arguments, and the communications will be done under the hood in the communications library, which will call the actual MPI routines. This approach will make the code more readable.

With this approach, we will make the program flexible too. For instance, if some of the communications routines can benefit from another library (e.g., SHMEM) for further efficiency, we can make that change in the communications library without making any changes to the main function.

3.2 Library for core computational routines

We will create a separate library for the core computational routines of the percolate program. This library will contain a routine for updating the cells (i.e., the while loop in the main function), and a routine for testing percolation.

This approach will enable us to use different versions of the while loop (e.g., one with fixed number of steps, and one where the algorithm stops when there are no further changes). Also, the separate routines in a library will declutter the main function.

3.3 Other possible changes

Although not mission critical, we may make further changes to our structure to make our code more readable. For instance, several lines of code in the main function are for initialising and copying arrays. These lines of code can also be abstracted away into routines in the library discussed in Section 3.2.

3.4 Concept map

We provided a concept map in Figure 1 to visualise the essential components of our program described in the above subsections.

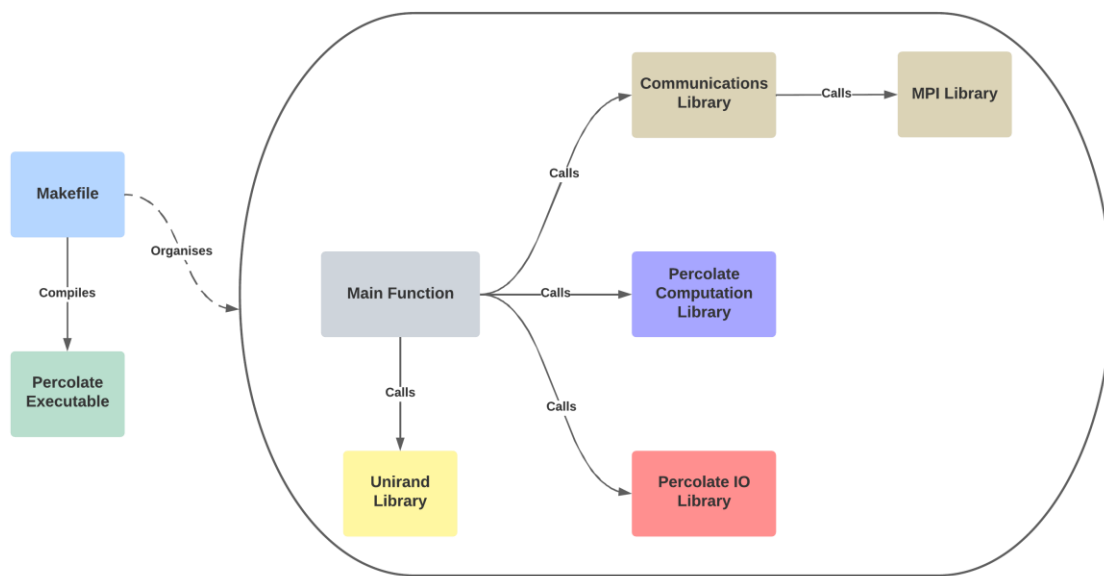


Figure 1: Concept map visualising the essential components of the percolate program.

4. Testing

In this section, we detail our method to test the program for both correctness and performance.

4.1 Correctness testing

For correctness testing, we will compare the output of the working serial code with that of the parallel code. We will run both programs with the same configuration parameters (i.e., ρ , seed), and then compare both the printed output and the map image.

Here is the list of data that we will compare to confirm correctness:

- (1) Compare that the actual density is equal for both serial and parallel versions.
- (2) Compare that the algorithm takes the same number of steps to complete in both versions.
- (3) Verify whether the given configuration parameters result in the same answer (i.e., whether percolation occurs or not) in both versions.
- (4) Compare the cluster sizes of the largest 8 clusters in both versions.
- (5) Visually inspect the largest 8 clusters from the output image in both versions.

We have conducted initial tests to develop a testing infrastructure that we can use in our second phase of the project. We selected two different configurations of ρ . In one configuration (i.e., ρ 0.4040), largest cluster does not percolate in the serial code. In another value of ρ (i.e., 0.4039), largest cluster percolates. We have carried out the steps outlined above, and we observed that the parallel code's output is the same as the output of the serial code. For simplicity, we visualised the images of the largest two clusters in Appendix A.

We are aware that implementing new boundary conditions with periodicity in the horizontal dimension will slightly change the output images, and the number of steps will not be the same as well. As long as our answer is correct (i.e., program correctly identifies whether the cluster percolates or not), any variances in the number of steps or output images can be ignored. Also, we will run the tests in different combinations of ρ and seed, so an incorrect code should reveal itself under different configuration parameters.

4.2 Performance testing

For performance tests, we plan to run the code across a range of process counts in Cirrus [2][3], extending beyond one node. We will calculate and visualise the speedup and the parallel efficiency of the parallel code, which are considered standard metrics in performance analysis of parallel codes. If the code is performant, we expect to see speedup increase as we increase the number of processes. Our code is not expected to scale linearly with the number of processes due to the inherent serial sections in the code, which does not benefit from parallelisation (e.g., initialisation, IO, reduction operations). This effect is expected from Amdahl's law.

Also, we will test our code across a range of system sizes (i.e., L) to analyse our code's scalability. We expect a performant code to improve in scalability with larger system sizes. This effect is expected from Gustafson's law.

Moreover, we will quote the serial efficiency in the second report. We need to measure this metric to detect whether there is a large variance between our serial code, and the parallel code run using one process.

We conducted initial performance tests in the ARCHER2 system [4] using our current parallel code using one-dimensional decomposition. In Figure 2, we observe that our code does not scale linearly with the number of processes due to the serial sections limiting scalability. Consequently, we observe in Figure 3 that the parallel efficiency declines with a higher number of processes. Moreover, both Figures 2 and 3 confirm that the code improves in scalability with a larger system size. In our second report, we will run these tests with higher process counts, and higher system sizes to analyse any further effects.

An alternative metric to measure parallel performance would be to compute average time per step (i.e., excluding serial sections). For this report, we did not compute this metric as we were interested in observing the effects of the serial sections of our parallel code. In our next report, we will measure this metric as well. Also, the tests were run only once per configuration. In our next report, we will run each configuration several times to reduce any noise from the system. With several runs, we can analyse the mean, standard deviation, and 95% confidence interval of runtimes.

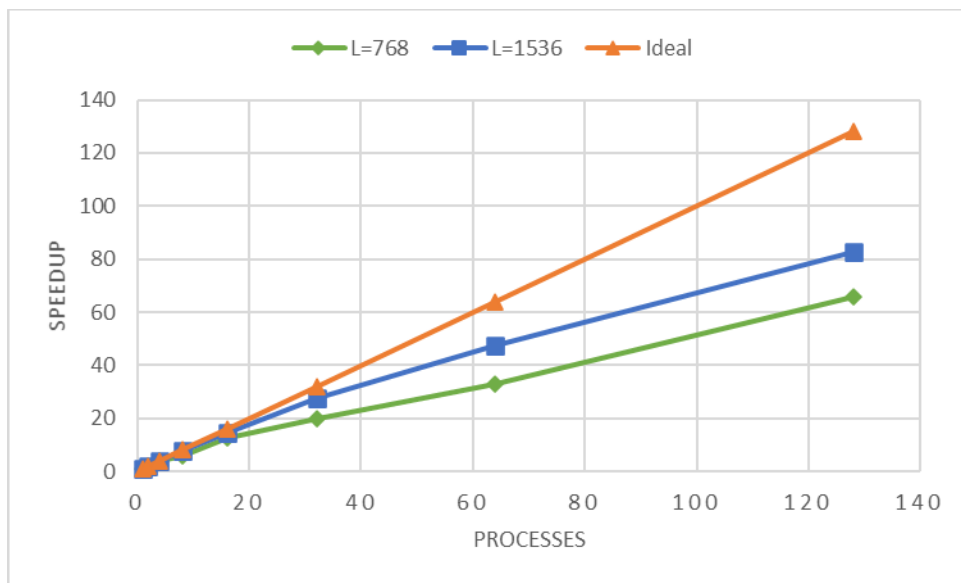


Figure 2: Speedup of the current version of parallel code.

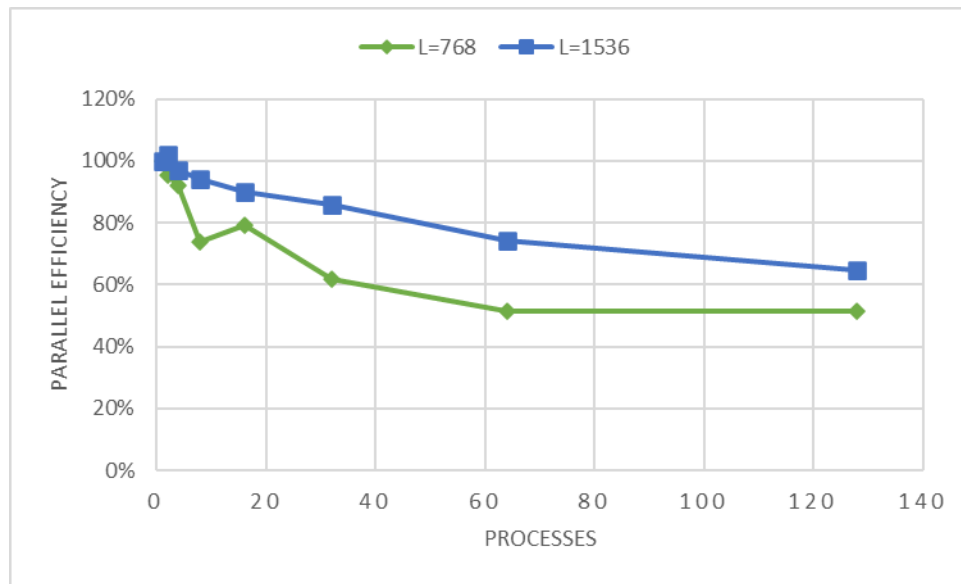


Figure 3: Parallel efficiency of the current version of parallel code.

5. Conclusion

The focus of this report was to discuss a plan to improve an existing parallel solution to the percolate problem using one-dimensional decomposition to a more advanced solution using two-dimensional decomposition. We discussed how we can use the various features of MPI to undertake communications. We detailed a structure of our program that will allow for flexibility and extensibility. Also, we discussed our method to test the program for correctness and performance along with initial results from the existing parallel code, which we can use as baseline reference. In the next report, we will discuss any design changes to the actual implementation, and the performance analysis of our improved code.

References

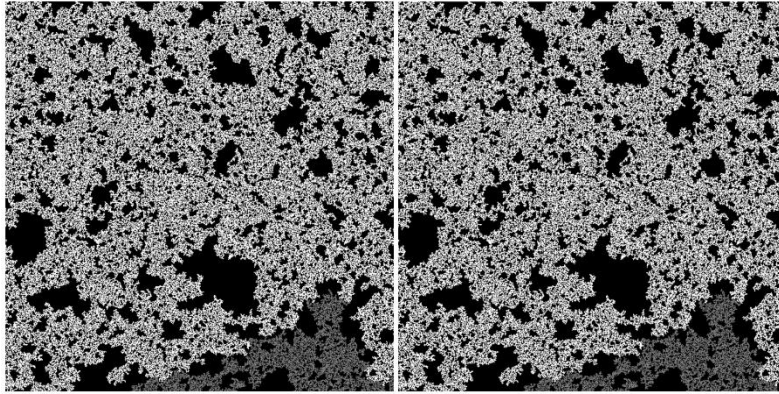
- [1] D. Henty. “MPI Casestudy: Parallel Percolation Simulation”, The University of Edinburgh, 2022.
- [2] EPCC. [Cirrus.readthedocs.io](https://cirrus.readthedocs.io). 2022. Cirrus — Cirrus 1.2 Documentation. [Online] Available: <https://cirrus.readthedocs.io/en/main/>. [Accessed: 20- Mar- 2022].

[3] EPCC. Cirrus Hardware. Cirrus.ac.uk, 2022. [Online]. Available: <https://www.cirrus.ac.uk/about/hardware.html>. [Accessed: 20- Mar- 2022].

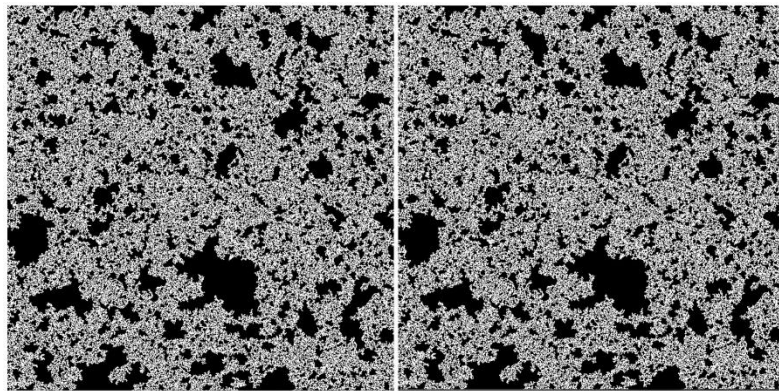
[4] EPCC. ARCHER2 Hardware & Software. Archer2.ac.uk. [Online]. Available: <https://www.archer2.ac.uk/about/hardware.html>. [Accessed: 20- Mar- 2022].

Appendix A: Correctness Tests

In this appendix, we include the output images of our initial correctness tests of the parallel code using one-dimensional decomposition.



(a) Both versions do not percolate (rho 0.4040).



(b) Both versions percolate (rho 0.4039).

Figure 4: Output images from sample correctness testing runs. The images on the left are of the serial code and the ones on the right are of the parallel code.